NCTU-EE ICLAB – Fall 2026

Lab05 Exercise:

Design: JPEG-Like-ENCoder **(JPEGLENC)**

Data Preparation

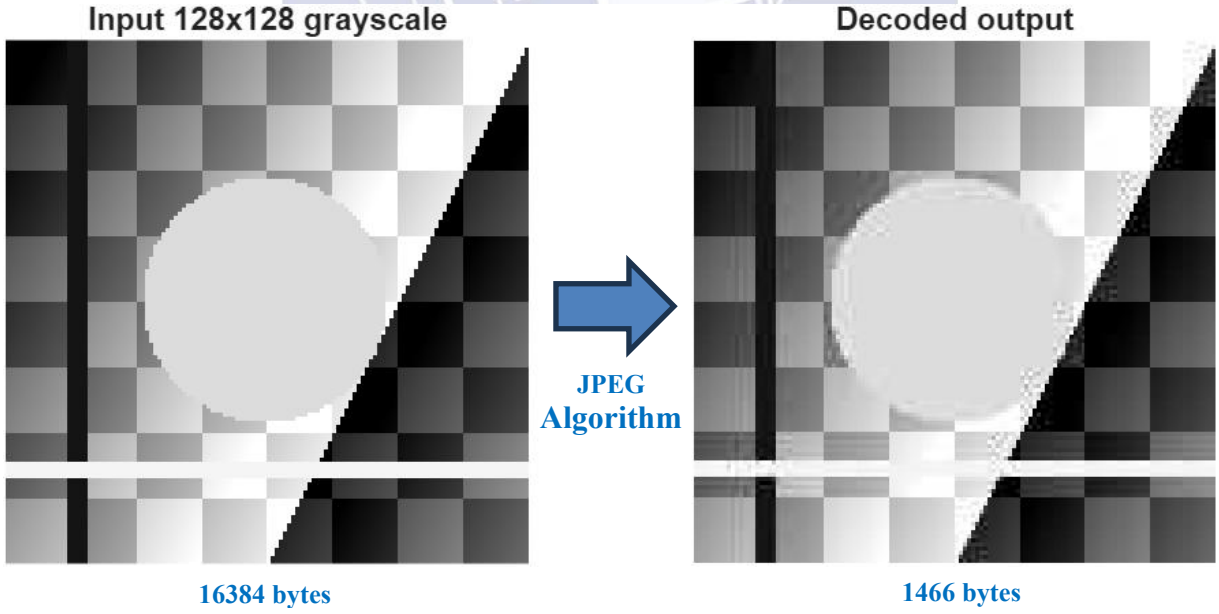
- Extract the test data from TA's directory:
  % tar xvf ~iclabTA01/Lab05_2026F.tar

- The extracted LAB directory contains:
  - Practice/
  - Exercise/

Design Description

Image compression reduces the storage and transmission bandwidth required for digital images. Neighboring pixels in a natural image are usually correlated, so a spatial block can be transformed into a small number of important low-frequency coefficients and many small or zero high-frequency coefficients. The remaining information can then be represented efficiently with differential coding, run-length coding, and variable-length Huffman codes.

In this exercise, you are asked to design a JPEG-like grayscale image encoder named JPEGLENC. The input is a single 128 x 128 unsigned 8-bit image. The encoder receives one pixel per cycle, divides the image into 256 non-overlapping 8 x 8 blocks, performs an integer two-dimensional DCT and power-of-two quantization, reorders the coefficients in zigzag order, and outputs a variable-length stream of raw entropy bytes. The 16,384 bytes to 1,466 bytes picture below is only an illustration. The output contains only the entropy-coded bytes: no JPEG headers or markers (SOI, DQT, DHT, ...) and no 0xFF/0x00 byte stuffing.



▲Example

Complete Algorithm Flow

The required behavior for one image can be summarized by the following ordered operations. These are algorithmic requirements:

- Accept 16,384 pixels in raster order and retain enough data to form each 8 x 8 block.
- Process blocks in block-raster order: block 0 at the upper-left corner, then left to right, and finally top to bottom through block 255 (Refer to Fig 2.).
- Subtract 128 from each pixel to obtain signed samples in the range -128 to 127.
- Apply the integer 2-D DCT, then apply signed rounding equivalent to division by 2^18 (round_s with n = 18) after the complete accumulation.
- Quantize every DCT coefficient D(u,v) with the specified shift matrix and the same symmetric signed rounding rule, Q(u,v) = round_s(D(u,v), S(u,v)); this is a second rounding, applied after the first one (Refer to Fig 4.).
- Read the 64 quantized coefficients using the specified zigzag order.
- Encode the DC difference, then encode the 63 AC coefficients with run-length symbols, EOB, and ZRL as required.
- Look up the fixed luminance Huffman code for each symbol and append its amplitude field immediately afterward, MSB first.
- Pack the continuous field sequence into bytes. Do not flush or pad at any 8 x 8 block boundary.
- After block 255, emit all complete bytes. If one to seven bits remain, append 1 bits to form one final byte.

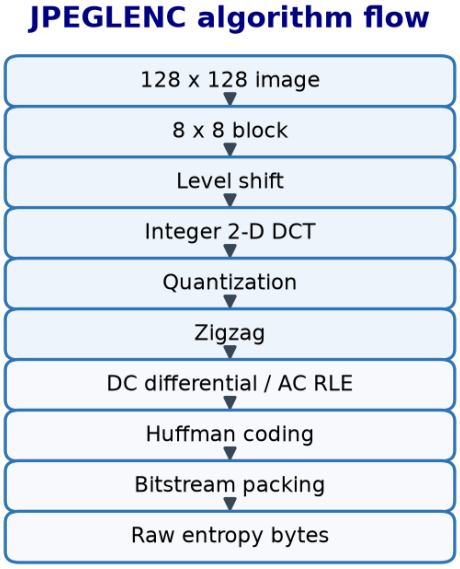


Fig 1. JPEG-Like ENCoder flow

Input image and block organization

Let x denote the horizontal pixel coordinate and y denote the vertical pixel coordinate. Both coordinates start at zero. Pixels arrive in raster-scan order: x increases from 0 to 127 within a row, then y increases by one. The one-dimensional input index is k = 128y + x, so the first three pixels are P(0,0), P(1,0), and P(2,0), and the first pixel of the second row is P(0,1) at k = 128.

The frame contains 16 blocks horizontally and 16 blocks vertically. For block coordinates bx and by, where 0 <= bx,by <= 15, the block ID is b = 16by + bx. A sample at local block coordinates lx and ly, where 0 <= lx,ly <= 7, is P(8bx + lx, 8by + ly). All 64 samples of block b must be transformed together before its entropy symbols are generated.

Quantity	Definition	Range
Input pixel index k	k = 128y + x	0 to 16,383
Block ID b	b = 16 floor(y/8) + floor(x/8)	0 to 255
Block coordinate	bx = b mod 16, by = floor(b/16)	0 to 15
Local coordinate	lx = x mod 8, ly = y mod 8	0 to 7

Table 1. Coordinate and ordering definitions

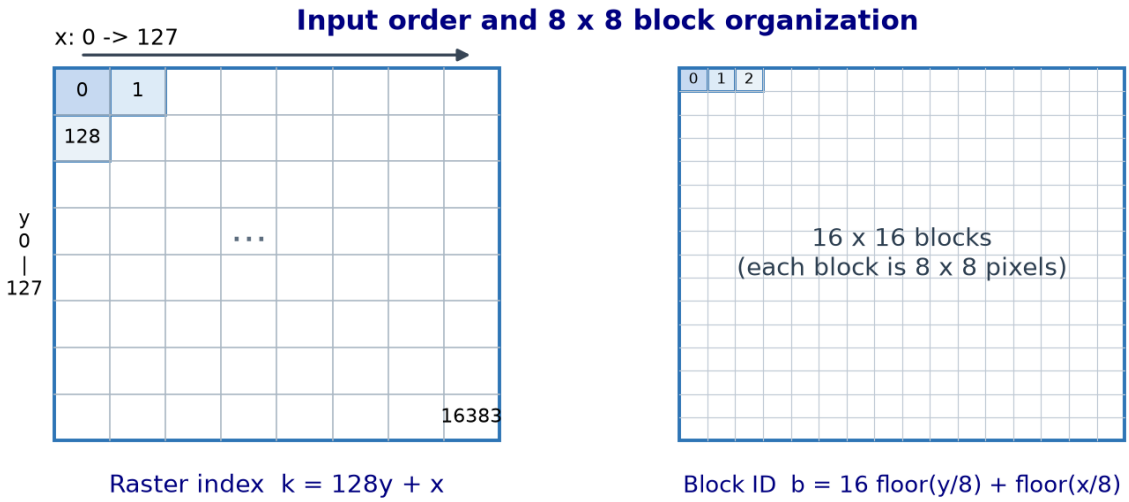


Fig 2. Raster input order and 8 x 8 block order

Level shift and 2-D DCT

Before transformation, every unsigned pixel is level-shifted by subtracting 128. This converts the range 0 to 255 into the signed range -128 to 127 and centers the block around zero.

$$P'(x,y) = P(x,y) - 128, \quad 0 \le P(x,y) \le 255, where\ P(x,y)\ is\ a\ pixel\ of\ the\ 8x8\ block.$$

Input pixel P	Level-shifted P'	Comment
0	-128	Minimum input
127	-1	One below the center
128	0	Centered value
255	127	Maximum input

Table 2. Level-shift examples

The 8 × 8 level-shifted pixel block is transformed into 64 frequency-domain coefficients using a two-

dimensional Discrete Cosine Transform (DCT). To avoid floating-point arithmetic, the cosine coefficients used in this exercise are represented by the fixed integer matrix C (refer to Table 3). Each entry of C is obtained by scaling the corresponding normalized cosine coefficient by 2^8 = 256 and converting the result to an integer.

The 2-D DCT can be expressed using an intermediate result (R(u,y)):

$$R(u,y) = \sum_{x=0}^{7} P'(x,y)C(u,x)$$

$$A(u,v) = \sum_{y=0}^{7} R(u,y)C(v,y)$$

Since two coefficients from (C) are used in each 2-D DCT calculation, the integer representation introduces a total scale factor of

$$2^8 \times 2^8 = 2^{16}.$$

In addition, the normalized 2-D DCT contains a factor of (1/4=1/2^2). Therefore, the accumulated value (A(u,v)) must be divided by

$$2^{16} \times 2^2 = 2^{18}$$

to obtain the final DCT coefficient. Thus, the signed rounding function is applied with **(n=18)**:

$$D(u,v) = round_s(A(u,v), 18)$$

where

$$round_s(a,n) = \begin{cases} (|a| + 2^{n-1}) >> n, & a \geq 0 \\ -((|a| + 2^{n-1}) >> n), & a < 0 \end{cases}$$

For the DCT calculation in this exercise, **n is always equal to 18**.

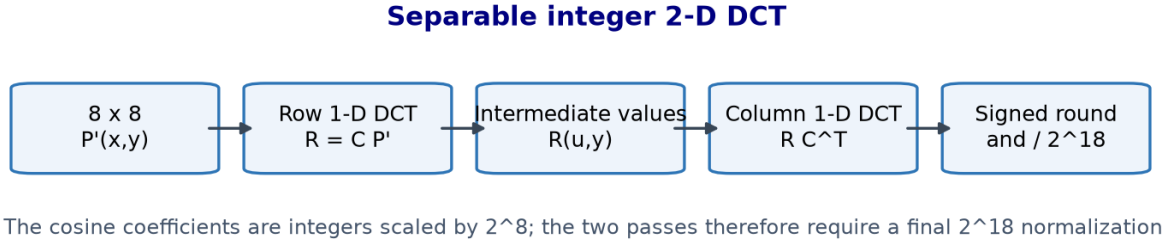


Fig 3. Integer 2-D DCT calculation flow

Do not round or truncate any intermediate value during the DCT accumulation. The DCT rounding

is applied only once, after the complete value A(u,v) has been calculated, and gives D(u,v); the quantization step then applies its own rounding to D(u,v). Early rounding may change the quantized coefficients and therefore produce a different final entropy stream.

Integer DCT coefficient matrix

C(u,x)	0	1	2	3	4	5	6	7
0	181	181	181	181	181	181	181	181
1	251	213	142	50	-50	-142	-213	-251
2	237	98	-98	-237	-237	-98	98	237
3	213	-50	-251	-142	142	251	50	-213
4	181	-181	-181	181	181	-181	-181	181
5	142	-251	50	213	-213	-50	251	-142
6	98	-237	237	-98	-98	237	-237	98
7	50	-142	213	-251	251	-213	142	-50

Table 3. Integer cosine coefficient matrix C

C is scaled by 2^8; the corresponding normalized coefficient is C(u,x)/2^8.

Exact symmetric signed rounding

The function round_s(a,n) divides a signed integer a by 2^n and rounds the magnitude to the nearest integer. A halfway magnitude is rounded away from zero. Implement this by taking the absolute value, adding 2^(n-1), shifting right by n, and restoring the original sign. Do not simply add a positive bias to a negative two's-complement number before an arithmetic shift.

Operation	Calculation	Result
round_s(25,3)	(25 + 4) >> 3 = 29 >> 3	+3
round_s(-25,3)	-((25 + 4) >> 3)	-3
round_s(28,3)	(28 + 4) >> 3 = 32 >> 3	+4
round_s(-28,3)	-((28 + 4) >> 3)	-4
round_s(-1024,4)	-((1024 + 8) >> 4)	-64

Table 4. Symmetric rounding examples

Power-of-two quantization and zigzag scan

Each DCT coefficient is quantized with a location-dependent power-of-two divisor. The fixed shift value S(u,v) ranges from 3 to 7. Low-frequency locations use smaller shifts and retain more precision, while high-frequency locations use larger shifts and are more likely to become zero, which improves compression efficiency. The same round_s function used by the DCT normalization must be used here with n = S(u,v). The quantizer input is the already rounded DCT coefficient D(u,v):

Q(u,v) = round_s(D(u,v), S(u,v))

These are two separate roundings. Do not merge them into round_s(A(u,v), 18 + S(u,v)); the result is different. Example with A = 917504 and S = 3: round_s(round_s(917504, 18), 3) = round_s(4, 3) =

1, but round_s(917504, 21) = 0.

Worked numeric example (constant block). If all 64 pixels of block 0 are 200, then P' = 72 everywhere. Every R(0,y) = 72 x 8 x 181 = 104,256, and every R(u,y) with u >= 1 is 0 because each of those rows of C sums to zero. Therefore A(0,0) = 104,256 x 8 x 181 = 150,962,688 and all other A(u,v) are 0. D(0,0) = round_s(150,962,688, 18) = 576, and with S(0,0) = 4, Q(0,0) = round_s(576, 4) = 36; all 63 AC coefficients are 0. With previous_dc = 0 this block is encoded as DC_LUT[6] = 1110, amplitude(36) = 100100, followed by EOB AC_LUT[8'h00] = 1010.

$$Q(u, v) = \text{round}_s(D(u, v), S(u, v))$$

After quantization, form a one-dimensional sequence Z[0] through Z[63]. Z[0] is the DC coefficient and Z[1] through Z[63] are AC coefficients. The required order is the order shown below; do not flatten the matrix in row-major order or rely on a language's default matrix memory order.

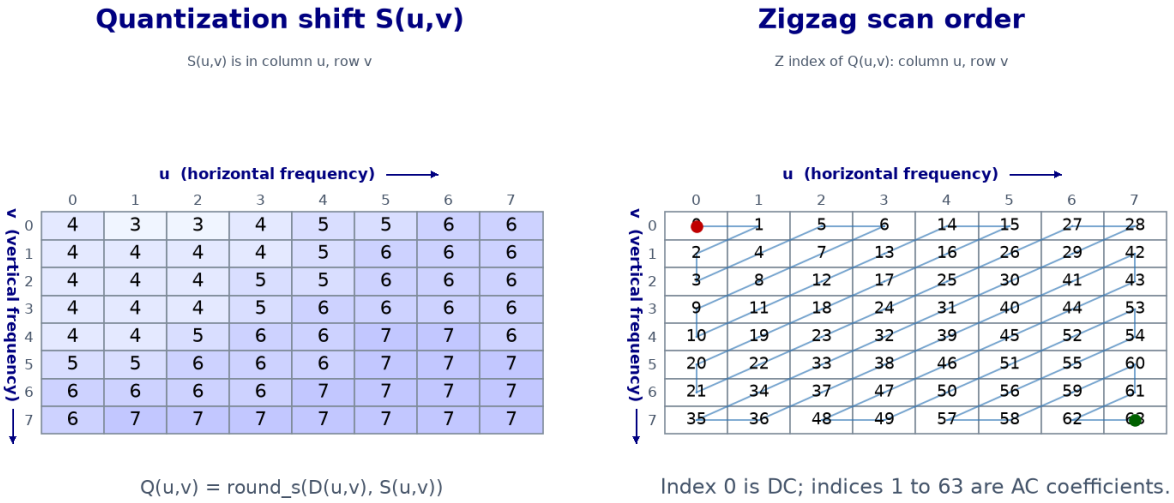


Fig 4. Quantization shift matrix and zigzag scan order

Reading check for Fig 4. Both tables are indexed by column u and row v, so S(1,0) = 3 and S(0,1) = 4, and the zigzag sequence starts with Z[0] = Q(0,0), Z[1] = Q(1,0), Z[2] = Q(0,1), Z[3] = Q(0,2). The same two tables are provided as text in 00_TESTBED: shift_table.txt gives S for every (U, V), one line each, and zigzag_table.txt gives the (U, V) of every zigzag INDEX, so that Z[INDEX] = Q(U, V). Read or copy these files instead of typing the numbers from the figure.

DC differential, AC run-length, and Huffman coding

Encoding summary:
DC: delta_DC -> SIZE -> DC Huffman(SIZE) -> amplitude
AC: zero RUN + SIZE -> AC Huffman(RUN,SIZE) -> amplitude
EOB: all remaining AC coefficients are zero
ZRL: 16 consecutive zeros before a later nonzero AC coefficient

The DC predictor previous_dc is initialized to zero before block 0 of every image. For each block,

calculate delta_dc = Z[0] - previous_dc, then update previous_dc = Z[0]. The predictor is not reset at row or block boundaries. There is no separate start-of-image port; each image transaction contains exactly 16,384 valid input pixels, and the next transaction begins a new image.

For any signed value a, size is the number of magnitude bits. If a is zero, size is zero and there is no amplitude field. If a is positive, its size-bit amplitude is its ordinary binary value. If a is negative, its amplitude is (2^size - 1) + a. This mapping is the ones-complement form used by JPEG; it is not the ordinary two's-complement representation of a. When delta_dc = 0, the DC code for SIZE 0 (00) is still emitted; only the amplitude field is absent.

$$\Delta DC = Z_0 - DC_{prev}, \qquad DC_{prev} \leftarrow Z_0$$

$$size(a) = 0 \ (a = 0), \quad size(a) = \lfloor \log_2 |a| \rfloor + 1 \ (a \neq 0)$$

$$amplitude(a, s) = a \ (a \geq 0), \quad amplitude(a, s) = (2^s - 1) + a \ (a < 0)$$

$$AC\ symbol = (run \ll 4) \mid size, \qquad EOB = 00_{16}, \quad ZRL = F0_{16}$$

Size	Positive range	Negative range	Amplitude example
0	0 only	--	No amplitude bits
1	1	-1	-1 -> 0
2	2 to 3	-3 to -2	-3 -> 00, -2 -> 01
3	4 to 7	-7 to -4	-7 -> 000, -4 -> 011
4	8 to 15	-15 to -8	-15 -> 0000, -8 -> 0111
5	16 to 31	-31 to -16	-25 -> 00110
6	32 to 63	-63 to -32	-32 -> 011111
7	64 to 127	-127 to -64	-64 -> 0111111
8 to 11	2^(s-1) to 2^s-1	-(2^s-1) to -2^(s-1)	Use the same mapping

Table 5. Size categories and negative-amplitude representation

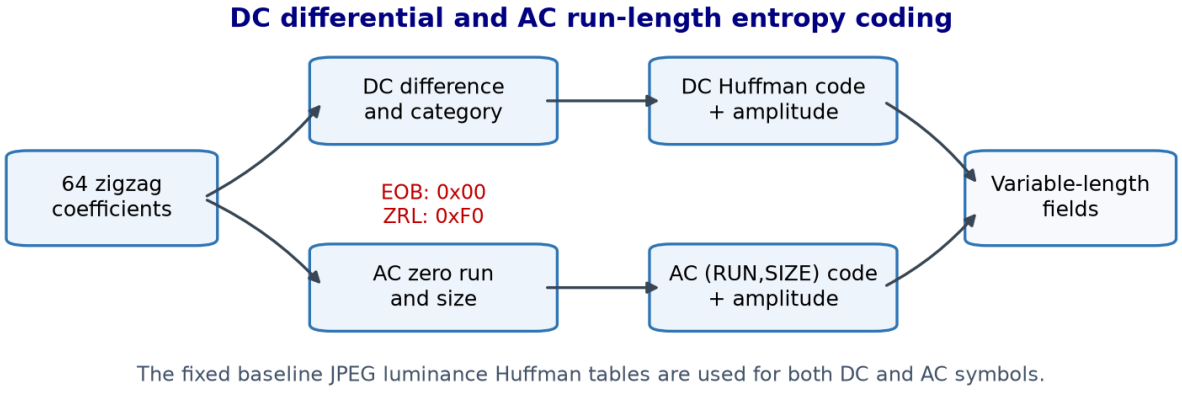


Fig 5. DC and AC entropy-coding flow

AC run-length algorithm

Scan Z[1] through Z[63] in order and maintain a zero counter run. When Z[k] is zero, increase run and continue. When a nonzero coefficient a is reached, first emit one ZRL symbol for every complete group of 16 zeros, reducing run by 16 each time. Then calculate the size of a, emit the Huffman code for symbol (run << 4) | size, and emit the size-bit amplitude of a. Finally set run back to zero.

If the scan reaches index 63 with one or more trailing zeros, emit one EOB symbol and stop the block. Do not emit ZRL symbols for groups that are entirely inside the final trailing-zero region; one EOB represents all remaining zeros. If Z[63] is nonzero, its normal RUN/SIZE symbol and amplitude finish the block and no EOB is appended.

Example: if Z[48] through Z[63] are zero and no later nonzero AC coefficient exists, emit one EOB only. Do not emit ZRL for these sixteen trailing zeros.

Case	Symbol	Required behavior
End of block	00 (EOB)	Emit once when all remaining AC positions are zero.
16 zeros before a later nonzero	F0 (ZRL)	Emit once per complete 16-zero group; then continue with the remaining run.
Nonzero AC	(RUN << 4) | SIZE	Emit its Huffman code, then exactly SIZE amplitude bits.
Last AC is nonzero	RUN/SIZE	Do not append EOB after coefficient 63.

Table 6. AC symbol rules

```
run = 0
for k = 1 ... 63:
    if Z[k] == 0: run = run + 1; continue
    while run >= 16:
        emit_huffman(F0);
        run = run - 16;
    s = size(Z[k]);
    emit_huffman((run << 4) | s)
    emit_amplitude(Z[k], s);
    run = 0

after the loop:
    if run > 0, emit_huffman(00)  // one EOB
```

Pseudo code for *AC run-length algorithm*

Huffman Coding and Bitstream Packing

The complete DC and AC Huffman lookup tables are provided in dc_huffman_lut.txt and ac_huffman_lut.txt. Use SIZE as the lookup address for dc_huffman_lut.txt, and use (RUN << 4) | SIZE as the lookup address for ac_huffman_lut.txt. Both files are in 00_TESTBED. The first line of each file is a header; every following line gives the address (CATEGORY, or SYMBOL in hex with its RUN and SIZE), the code LENGTH, and the code itself in hex (CODE_HEX, right aligned) and in binary (CODE_BITS, MSB first).

Example blocks. 00_TESTBED/example_trace.txt follows the first three blocks of an example image step by step: the pixels, D(u,v), Q(u,v), the zigzag sequence Z[0..63], every encoded field (Huffman code and amplitude bits) and the resulting bytes. Example block 0 is the first block of the image (previous_dc = 0) and gives Z[0] = -20. Example block 1 is exactly the worked example of

Table 8 / Fig 6 (previous_dc = -20, Z[0] = -15, Z[3] = +3, Z[22] = -2), so its fields are the same 41 bits as in Fig 6. Example block 2 is a busy block that continues the same bitstream. Compare your intermediate values with this file to find out which step is wrong before checking the whole bitstream.

For each encoded coefficient, append the Huffman code first, followed immediately by its amplitude bits if an amplitude field exists. Both fields are appended MSB first.

All Huffman codes and amplitude fields from all 256 blocks are concatenated into one continuous bitstream. No delimiter or padding is inserted between fields or between 8 × 8 blocks.

Whenever eight consecutive bits are available, they form one output byte. The earliest bit in the bitstream becomes out_data[7], and the eighth bit becomes out_data[0].

For example,

Bitstream: 110 01011 1010 100 1 ... (fields)

DC_size LUT | DC_amp | EOB | DC_size LUT

11001011 10101001 ...(bytes)

Output bytes: CB A9 ...

The internal implementation of the bit-packing process is unrestricted, as long as the generated byte stream exactly matches the specified bitstream.

Item	Symbol/category	Length	Code bits
DC example	category 0	2	00
DC example	category 6	4	1110
DC example	category 7	5	11110
AC EOB	00	4	1010
AC ZRL	F0	11	11111111001

Table 7. Representative entries from the fixed luminance Huffman tables

Block Boundary Rule

The bitstream continues directly across 8 × 8 block boundaries. Do not insert padding or force byte alignment after a block.

The DC predictor previous_dc also continues across block boundaries and is reset only at the beginning of a new image or by reset.

End-of-image padding

After block 255 has been completely encoded, output all complete bytes. If no bits remain, encoding is complete. If 1 to 7 bits remain, append logic-1 bits to the end of the bitstream until eight bits are formed, then output the final byte.

Worked Entropy-Coding Example

Assume previous_dc = -20 and the current quantized zigzag sequence has Z[0] = -15, Z[3] = +3, and

Z[22] = -2; all other AC coefficients are zero. In other words, two zeros appear before +3, eighteen zeros appear between +3 and -2, and every coefficient after -2 is zero. The eighteen-zero run is important: one ZRL consumes the first sixteen zeros, while the remaining two zeros must still be included in the RUN/SIZE symbol for -2.

Step	Calculation / LUT address	Generated bits	Key idea
DC prediction	delta_dc = -15 - (-20) = +5; size = 3	DC_LUT[3] = 100; amplitude(+5) = 101	Update previous_dc to -15 after forming the difference.
First nonzero AC	Z[1]=Z[2]=0, Z[3]=+3; run=2, size=2, symbol=8'h22	AC_LUT[8'h22] = 11111001; amplitude(+3) = 11	A normal AC symbol contains both zero run and coefficient size.
Long zero run	Z[4] through Z[21] are 18 zeros before Z[22]	AC_LUT[8'hF0] = 11111111001	ZRL consumes 16 zeros; two zeros remain in run.
Residual run + negative AC	Z[22]=-2; run=2, size=2, symbol=8'h22	AC_LUT[8'h22] = 11111001; amplitude(-2) = 01	The two zeros left after ZRL become the RUN field of the next symbol.
Trailing zeros	Z[23] through Z[63] are all zero	AC_LUT[8'h00] = 1010	One EOB replaces all remaining trailing zeros.
Byte packing	41 bits are concatenated MSB first	97 E7 FF 3F 2D, with one bit 0 remaining in the bitstream	The block is not byte-aligned, and the raw FF is not followed by a stuffed 00.

Table 8. Entropy coding example containing RUN/SIZE, ZRL, negative amplitude, and EOB

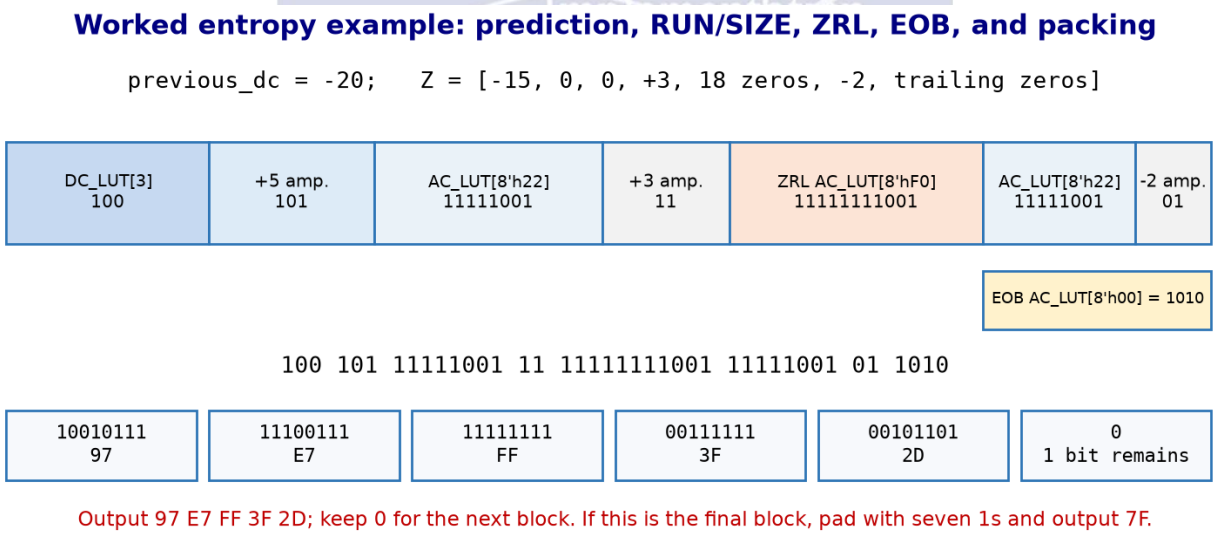


Fig 6. Field concatenation and byte packing for the worked entropy-coding example

The complete field sequence is 100 101 11111001 11 11111111001 11111001 01 1010, for a total of 41 bits. The bitstream produces the complete bytes 97, E7, FF, 3F, and 2D, with one final bit 0 remaining.

If this is not the last block, the next block's DC Huffman code is appended immediately after the remaining 0 bit. No padding occurs at the block boundary. If this is block 255, append seven logic ones to form 01111111 and output the final byte 7F. This distinction is the reason the entropy bitstream must remain continuous across block boundaries.

The flow for input/output

After reset, one image is supplied by asserting in_valid for exactly 16,384 consecutive cycles. The pixel on in_data is valid on every one of those cycles; there are no input bubbles. The first valid pixel is image coordinate (0,0), and the last is (127,127). A new image must not begin while the preceding image is still producing output.

Output is permitted only after the last input pixel has been accepted. Because transform and entropy-coding schedules may differ, out_valid may contain bubbles. Every high out_valid cycle transfers exactly one encoded byte. The interface has no out_last port, so the encoder simply stops asserting out_valid after emitting the final padded byte.

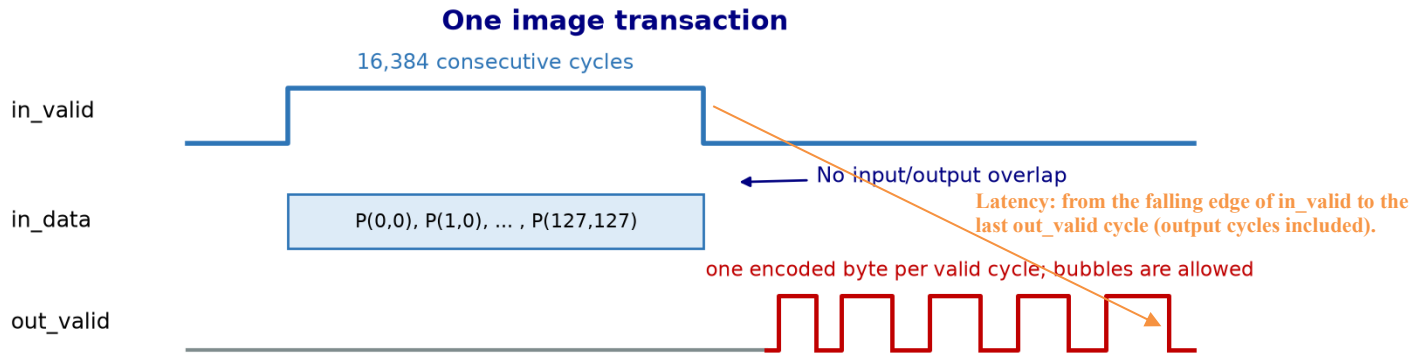


Fig 7. Input/output flow for one image

I/O Description

Input	Bit Width	Definition and Description
clk	1	Clock signal.
rst_n	1	Asynchronous active-low reset.
in_valid	1	High when in_data carries a valid input pixel. It is asserted for 16,384 consecutive cycles for one image.
in_data	8	Unsigned grayscale pixel in raster-scan order, from P(0,0) to P(127,127).
Output	**Bit Width**	**Definition and Description**
out_valid	1	High when out_data is a valid encoded byte. Bubbles are allowed; it must not overlap in_valid.
out_data	8	One raw entropy byte, MSB first. It must be 0 whenever out_valid is low.

Specification

Top module

1. Top module name: **JPEGLENC**(file name: **JPEGLENC.v**)

2. You can adjust your clock period by yourself, but the maximum period is 20 ns. The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.

3. The execution latency is limited to 50000 cycles. The latency is the clock cycles between the falling edge of the last cycle of **in_valid** and the last out_valid cycle, and the cycles in which out_valid is high are included; the 16,384 input cycles are not counted. For example: in_valid falls at negative edge E1 (counted as cycle 1), and the image produces 3 bytes, sampled at E5, E6 and E8; the latency of this image is 8 cycles.

4. The total cell area should not be larger than 2,000,000 μm^2.

5. In this lab, you must use the memory and generate it yourself. The number of words and the bits per each word is defined by yourself. The total number and kind of memory is unlimited. We will check it at **JPEGLENC.area in 02_SYN/Report**. The area of **Macro/Black Box** must not be 0. The example is shown in following figure.

```
Combinational area:            345277.901094
Buf/Inv area:                   16052.097460
Noncombinational area:         236478.618393
Macro/Black Box area:          906181.562500
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:              1487938.081987
```

6. There should be no MEM_error allowed in the simulation, and when having memory errors, you will fail this lab.

```
71    ** MEM_Error: Unknown value occurred (          118750 ps) in Address of TESTBED.u_MVDM.L0.ErrorMessage
72    ** MEM_Error: Unknown value occurred (          118750 ps) in Address of TESTBED.u_MVDM.L1.ErrorMessage
```

7. If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message. Even though the design may still pass gate level simulation, this situation will be regarded as a synthesis fail. In this case, the memory area will be 0 in **JPEGLENC.area**. We will check it at **syn.log** and **JPEGLENC.area**.

Reset

8. It is asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.

9. The reset signal (**rst_n)** will be given only once at the beginning of simulation. All output signals should be reset to 0 after the reset signal is asserted.

in_valid & input signals

10. All input signals are synchronized at negative edge of the clock.

11. **in_valid** will come after reset.

12. **All the input data is sent by in_data for 128 x 128 input image.**

13. The input of **image** is delivered in raster scan order for 128×128 cycles continuously when **in_valid** is tied high. When all elements of the images are delivered completely, **in_data** is tied to unknown state, and **in_valid** will also be tied low.

14. In each pattern, **in_valid** will come in 3~6 negative edge of clock after last **out_valid** falls.

out_valid & out_data

15. All output signals should be synchronized at clock positive edge.

16. The TA's pattern will capture your output for checking at clock negative edge.

17. **out_valid** cannot overlap with **in_valid** at any time.

18. **out_valid** is limited to be high only when **out_data** is valid.

19. **out_data** should be correct when **out_valid** is high. And **out_data** should be zero when **out_valid** is low.

20. The number of cycles in which **out_valid** is high must be exactly the number of encoded bytes of the image. Each of these cycles carries one byte on **out_data**, in bitstream order. These cycles do **not need to be consecutive**: **out_valid** may go low between two bytes (**bubbles are allowed**). Sending more or fewer bytes is regarded as a fail.

21. The next input pattern will come in 3~6 negative edge of clock after the last **out_valid** of this pattern finish.

Synthesis

22. The input delay is set to 0.5*(clock period).

23. The output delay is set to 0.5*(clock period), and the output loading is set to 0.05**.**

24. The input delay of **clk** and **rst_n** should be zero.

25. The synthesis time should be less than 2 hour.

26. Do not write comment in the filelist.f file, or the simulation may pass in your terminal but will FAIL in TA's.

```
1  TESTBED.v
2  //../04_MEM/L0_16index_ver_1024X128.v
3  //../04_MEM/L0_index8_2048X64.v
4  ../04_MEM/L0_index16_1024X128_SP.v
5  ../04_MEM/L1_index16_1024X128_SP.v
```

(NEVER do that)

27. The synthesis result (syn.log) of data type cannot include any latches and error.

28. After synthesis, you can check **JPEGLENC.area** and **JPEGLENC.timing**. The area report is

valid when the slack in the end of timing report should be non-negative and the result should be MET.

Gate-level simulation

29. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

Supplement

30. In this lab, you are NOT allowed to use Designware IP.

31. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

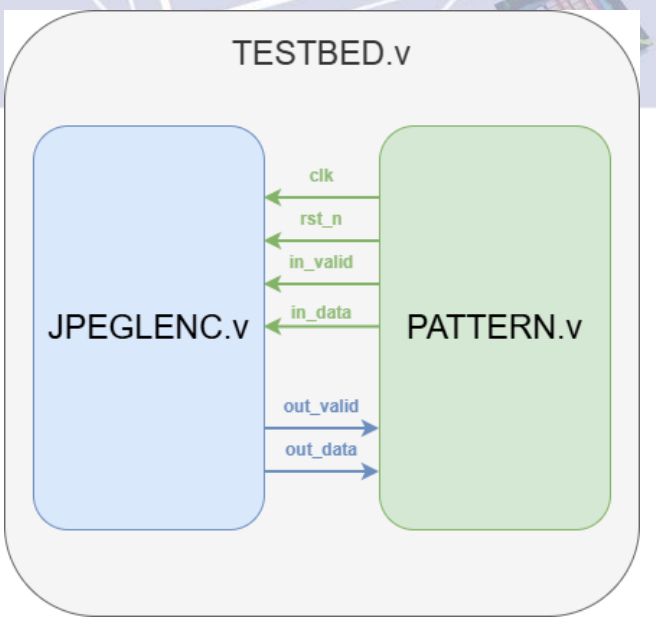
32. Don't write Chinese or other language comments in the file you sent.

33. Verilog command //synopsys dc_script_begin, //synopsys dc_script_end //synopsys translate_off, //synopsys translate_on are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.

34. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.

35. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your design. The less cost your design has, the higher grade you get.

- 2 demo: Score*0.7
- Function Validity: 70%
- Performance: 30%
  - $Area^2$ * Total latency
  - Total latency = cycle time * (sum of the execution latency of all patterns)

Note

- Please submit your design (JPEGLENC.v) in 09_SUBMIT folder
  - 1st_demo deadline : 2026/10/19 (Mon.) 12:00:00
  - 2nd_demo deadline: 2026/10/21 (Wed.) 12:00:00
- Please upload the following files under 09_SUBMIT
  - **filelist.f (must include all of your memory.v inside and do not add any comment)**
  - **JPEGLENC.v**
  - **./04_MEM folder (where you put all of used memory)**
    - <Memory name>.v
    - <Memory name>_WC.lib
    - <Memory name>_WC.db (The .db file must be the worst-case (WC) corner generated by memaker. The TA will check it; any mismatch fails the demo.)
  - In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.
  - If your files violate the naming rule, you will get 5 deduction points.
- Template folders and reference commands:
  - 01_RTL/ (RTL simulation) **./01_run_vcs_rtl**
  - 02_SYN/ (Synthesis) **./01_run_dc_shell**

- Check if there is any latch in your design in syn.log
- Check the timing of design in /Report/JPEGLENC.timing
- Check the area of design in /Report/ JPEGLENC.area

➢ 03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**

➢ 04_MEM/ (Memory location)

- You should generate your own memory and put the required files here, or you may fail the demo

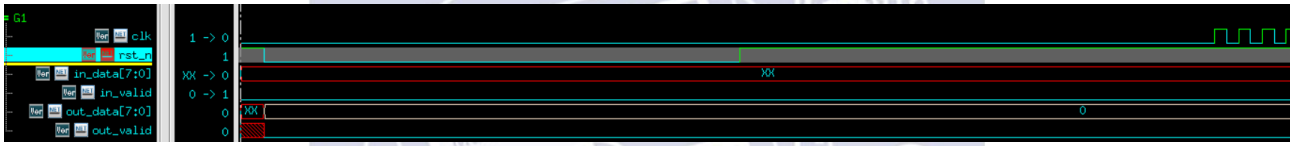
➢ 09_SUBMIT/ (Submit your files) **./00_tar**

./01_submit

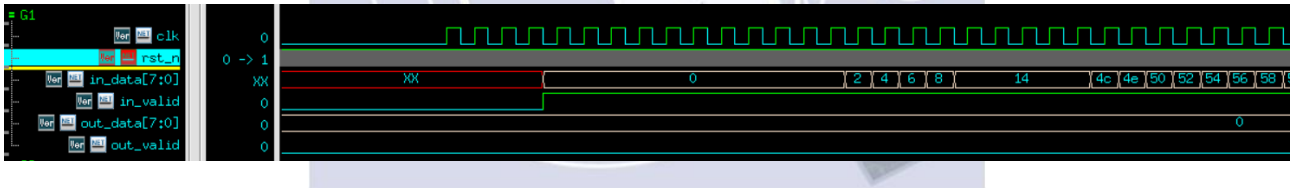
./02_check

Reference Waveform

- Asynchronous reset and active-low reset all output to low



- Input of in_data when in_valid is high(total 128*128 cycles)



- Output of out_data when out_valid is high

